Recurrent neural network based language model

Tomáš Mikolov[1,2], *Martin Karafiát*[1], *Lukáš Burget*[1], *Jan "Honza" Černocký*[1], *Sanjeev Khudanpur*[2]

[1]Speech@FIT, Brno University of Technology, Czech Republic
[2] Department of Electrical and Computer Engineering, Johns Hopkins University, USA
{imikolov,karafiat,burget,cernocky}@fit.vutbr.cz, khudanpur@jhu.edu

Abstract

A new recurrent neural network based language model (RNN LM) with applications to speech recognition is presented. Results indicate that it is possible to obtain around 50% reduction of perplexity by using mixture of several RNN LMs, compared to a state of the art backoff language model. Speech recognition experiments show around 18% reduction of word error rate on the Wall Street Journal task when comparing models trained on the same amount of data, and around 5% on the much harder NIST RT05 task, even when the backoff model is trained on much more data than the RNN LM. We provide ample empirical evidence to suggest that connectionist language models are superior to standard n-gram techniques, except their high computational (training) complexity.

Index Terms: language modeling, recurrent neural networks, speech recognition

1. Introduction

Sequential data prediction is considered by many as a key problem in machine learning and artificial intelligence (see for example [1]). The goal of statistical language modeling is to predict the next word in textual data given context; thus we are dealing with sequential data prediction problem when constructing language models. Still, many attempts to obtain such statistical models involve approaches that are very specific for language domain - for example, assumption that natural language sentences can be described by parse trees, or that we need to consider morphology of words, syntax and semantics. Even the most widely used and general models, based on n-gram statistics, assume that language consists of sequences of atomic symbols - words - that form sentences, and where the end of sentence symbol plays important and very special role.

It is questionable if there has been any significant progress in language modeling over simple n-gram models (see for example [2] for review of advanced techniques). If we would measure this progress by ability of models to better predict sequential data, the answer would be that considerable improvement has been achieved - namely by introduction of cache models and class-based models. While many other techniques have been proposed, their effect is almost always similar to cache models (that describe long context information) or class-based models (that improve parameter estimation for short contexts by sharing parameters between similar words).

If we would measure success of advanced language modeling techniques by their application in practice, we would have to be much more skeptical. Language models for real-world speech recognition or machine translation systems are built on huge amounts of data, and popular belief says that more data is all we need. Models coming from research tend to be complex and often work well only for systems based on very limited amounts of training data. In fact, most of the proposed advanced language modeling techniques provide only tiny improvements over simple baselines, and are rarely used in practice.

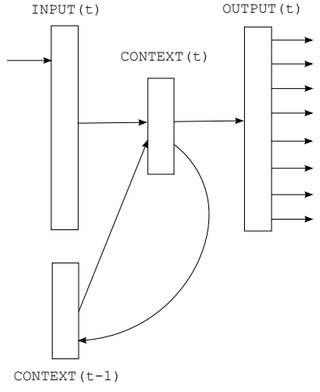

Figure 1: *Simple recurrent neural network.*

2. Model description

We have decided to investigate recurrent neural networks for modeling sequential data. Using artificial neural networks in statistical language modeling has been already proposed by Bengio [3], who used feedforward neural networks with fixed-length context. This approach was exceptionally successful and further investigation by Goodman [2] shows that this single model performs better than mixture of several other models based on other techniques, including class-based model. Later, Schwenk [4] has shown that neural network based models provide significant improvements in speech recognition for several tasks against good baseline systems.

A major deficiency of Bengio's approach is that a feedforward network has to use fixed length context that needs to be specified *ad hoc* before training. Usually this means that neural networks see only five to ten preceding words when predicting the next one. It is well known that humans can exploit longer context with great success. Also, cache models provide complementary information to neural network models, so it is natural to think about a model that would encode temporal information implicitly for contexts with arbitrary lengths.

Recurrent neural networks do not use limited size of context. By using recurrent connections, information can cycle in-

基于循环神经网络的语言模型

托姆·阿什·米科洛 $v^{1,2}$,马丁·卡拉菲·在[1],卢卡什·布尔格特[1], 扬·"霍恩扎"·切尔诺茨基·伊[1], 桑吉夫·库丹普尔处 [2]

[1]Speech@FIT，布尔诺技术大学，捷克共和国[2] 电气与计算机工程系，约翰斯·霍普金斯大学，美国{imikolov,karafiat,burget,cernocky}@fit.vutbr.cz, khudanpur@jhu.edu

摘要

本文介绍了一种全新的基于循环神经网络的语言模型（RNN大语言模型），并将其应用于语音识别领域。实验结果表明，与最先进的回退语言模型相比，通过使用多个循环神经网络语言模型的混合模型，可将语言模型的困惑度降低约50%。在《华尔街日报》任务中，当训练数据量相同时，该模型能使词错误率降低约18%；而在难度更高的NIST RT05任务中，即便回退语言模型所使用的训练数据量远多于该RNN大语言模型，其词错误率仍能降低约5%。我们通过大量实验证据表明，除了计算（训练）复杂度较高之外，联结主义语言模型仍优于标准n-gram技术。

索引术语：语言模型、循环神经网络、语音识别

1. 引言

在机器学习和人工智能领域，许多专家都将序列数据预测视为核心问题（例如可参考 [1]）。统计语言建模的目标是在给定上下文的情况下预测文本数据中的下一个单词；因此，在构建语言模型时我们实际上就是在解决序列数据预测问题。不过，目前众多用于构建此类统计模型的方法都高度依赖于语言领域的特定假设——比如认为自然语言句子可以用解析树来描述，或者认为必须考虑单词的形态学、句法及语义特征。即便是一些应用最广泛、最具通用性的模型，即基于n-gram统计的模型，也依然假定语言是由构成句子的原子符号——即单词——所构成的序列，而且句子结束标记在其中起着极为特殊且重要的作用。

相较于简单的n-gram模型，语言模型领域是否取得了显著进展仍值得商榷（相关高级技术综述可参见 [2]）。如果以模型预测序列数据的能力作为衡量标准，答案则是确实已经实现了相当大的提升——而这主要得益于缓存模型的引入。

此外还有基于类别的模型。尽管已有许多其他技术被提出，但它们的效果几乎总是与缓存模型（用于处理长上下文信息）或基于类别的模型（通过在相似词汇间共享参数来提升短上下文下的参数估计精度）类似。

如果以这些先进语言模型技术的实际应用情况来衡量其成功与否，我们恐怕必须持更为审慎的态度。用于现实场景中的语音识别或机器翻译系统的语言模型，都是建立在海量训练数据之上的，而普遍的看法认为只要数据足够多就足够了。那些源自研究领域的模型往往结构较为复杂，且通常只适用于那些依赖极少量训练数据的系统。事实上，大多数被提出的先进语言建模技术，其提升效果相较于简单的基准模型而言微乎其微，因此在实际应用中很少被采用。

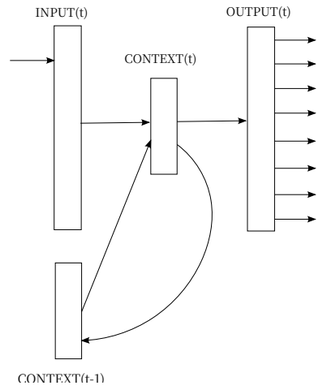

图1：简单循环神经网络。

这类技术很少在实践中得到应用，因为它们大多只能为基于少量训练数据的系统带来有限的性能提升。

2. 模型描述

我们决定研究循环神经网络在序列数据建模中的应用。Bengio [3], 早已提出在统计语言建模中运用人工神经网络，并采用了带有固定长度上下文的前馈神经网络，这一方法取得了极为显著的成效。Goodman [2] 的后续研究进一步表明，这种单一模型相较于基于其他技术、包括基于类别的模型构建的多种其他模型的混合模型，表现更为出色。后来，Schwenk [4] 也证明，基于神经网络的模型在多项语音识别任务中，相较于现有的优秀基准系统，能够实现显著的性能提升。

Bengio方法的明显缺陷在于，前馈网络必须使用在训练前需要临时指定的固定长度上下文。通常这意味着，在预测下一个词时，神经网络只能参考前五到十个词的信息。而众所周知，人类能够非常有效地利用更长的上下文信息。此外，缓存模型还能为神经网络模型提供补充信息，因此人们很自然地会考虑设计一种能够为任意长度的上下文隐式编码时间信息的模型。

循环神经网络无需受限于有限的上下文长度。借助循环连接机制，信息能够不断循环传递。

side these networks for arbitrarily long time (see [5]). However, it is also often claimed that learning long-term dependencies by stochastic gradient descent can be quite difficult [6].

In our work, we have used an architecture that is usually called a *simple recurrent neural network* or Elman network [7]. This is probably the simplest possible version of recurrent neural network, and very easy to implement and train. The network has an input layer x, hidden layer s (also called context layer or state) and output layer y. Input to the network in time t is $x(t)$, output is denoted as $y(t)$, and $s(t)$ is state of the network (hidden layer). Input vector $x(t)$ is formed by concatenating vector w representing current word, and output from neurons in context layer s at time $t-1$. Input, hidden and output layers are then computed as follows:

$$x(t) = w(t) + s(t-1) \tag{1}$$

$$s_j(t) = f\left(\sum_i x_i(t) u_{ji}\right) \tag{2}$$

$$y_k(t) = g\left(\sum_j s_j(t) v_{kj}\right) \tag{3}$$

where $f(z)$ is sigmoid activation function:

$$f(z) = \frac{1}{1+e^{-z}} \tag{4}$$

and $g(z)$ is softmax function:

$$g(z_m) = \frac{e^{z_m}}{\sum_k e^{z_k}} \tag{5}$$

For initialization, $s(0)$ can be set to vector of small values, like 0.1 - when processing a large amount of data, initialization is not crucial. In the next time steps, $s(t+1)$ is a copy of $s(t)$. Input vector $x(t)$ represents word in time t encoded using 1-of-N coding and previous context layer - size of vector x is equal to size of vocabulary V (this can be in practice $30\,000 - 200\,000$) plus size of context layer. Size of context (hidden) layer s is usually $30 - 500$ hidden units. Based on our experiments, size of hidden layer should reflect amount of training data - for large amounts of data, large hidden layer is needed[1].

Networks are trained in several epochs, in which all data from training corpus are sequentially presented. Weights are initialized to small values (random Gaussian noise with zero mean and 0.1 variance). To train the network, we use the standard backpropagation algorithm with stochastic gradient descent. Starting learning rate is $\alpha = 0.1$. After each epoch, the network is tested on validation data. If log-likelihood of validation data increases, training continues in new epoch. If no significant improvement is observed, learning rate α is halved at start of each new epoch. After there is again no significant improvement, training is finished. Convergence is usually achieved after 10-20 epochs.

In our experiments, networks do not overtrain significantly, even if very large hidden layers are used - regularization of networks to penalize large weights did not provide any significant improvements. Output layer $y(t)$ represents probability distribution of next word given previous word $w(t)$ and context $s(t-1)$. Softmax ensures that this probability distribution is valid, ie. $y_m(t) > 0$ for any word m and $\sum_k y_k(t) = 1$.

At each training step, error vector is computed according to cross entropy criterion and weights are updated with the standard backpropagation algorithm:

$$\text{error}(t) = \text{desired}(t) - y(t) \tag{6}$$

where $desired$ is a vector using 1-of-N coding representing the word that should have been predicted in a particular context and $y(t)$ is the actual output from the network.

Note that training phase and testing phase in statistical language modeling usually differs in the fact that models do not get updated as test data are being processed. So, if a new personname occurs repeatedly in the test set, it will repeatedly get a very small probability, even if it is composed of known words. It can be assumed that such long term memory should not reside in activation of context units (as these change very rapidly), but rather in synapses themselves - that the network should continue training even during testing phase. We refer to such model as *dynamic*. For dynamic model, we use fixed learning rate $\alpha = 0.1$. While in training phase all data are presented to network several times in epochs, dynamic model gets updated just once as it processes testing data. This is of course not optimal solution, but as we shall see, it is enough to obtain large perplexity reductions against static models. Note that such modification is very similar to cache techniques for backoff models, with the difference that neural networks learn in continuous space, so if 'dog' and 'cat' are related, frequent occurrence of 'dog' in testing data will also trigger increased probability of 'cat'.

Dynamically updated models can thus automatically adapt to new domains. However, in speech recognition experiments, history is represented by hypothesis given by recognizer, and contains recognition errors. This generally results in poor performance of cache n-gram models in ASR [2].

The training algorithm described here is also referred to as truncated backpropagation through time with $\tau = 1$. It is not optimal, as weights of network are updated based on error vector computed only for current time step. To overcome this simplification, backpropagation through time (BPTT) algorithm is commonly used (see Boden [5] for details).

One of major differences between feedforward neural networks as used by Bengio [3] and Schwenk [4] and recurrent neural networks is in amount of parameters that need to be tuned or selected *ad hoc* before training. For RNN LM, only size of hidden (context) layer needs to be selected. For feedforward networks, one needs to tune the size of layer that *projects* words to low dimensional space, the size of hidden layer and the context-length[2].

2.1. Optimization

To improve performance, we merge all words that occur less often than a threshold (in the training text) into a special *rare* token. Word-probabilities are then computed as

$$P(w_i(t+1)|w(t), s(t-1)) = \begin{cases} \frac{y_{rare}(t)}{C_{rare}} & \text{if } w_i(t+1) \text{ is rare,} \\ y_i(t) & \text{otherwise} \end{cases} \tag{7}$$

[1] Consequently, time needed to train optimal network increases faster than just linearly with increased amount of training data: vocabulary growth increases the input and output layer sizes, and also the optimal hidden layer size increases with more training data.

[2] It is out of scope of this paper to provide a detailed comparison of feedforward and recurrent networks. However, in some experiments we have achieved almost twice perplexity reduction over n-gram models by using a recurrent network instead of a feedforward network.

因此这类网络能够在任意长的时间跨度内进行运算（详见 [5]）。不过也有观点认为，通过随机梯度下降法来学习长期依赖关系其实相当困难 [6]。

在我们的研究中，我们采用了通常被称为简单循环神经网络或Elman网络的架构 [7]。这大概是循环神经网络最简单的形式，且实现与训练都非常容易。该网络包含输入层 x、隐藏层 s （也称为上下文层或状态层）以及输出层 y。在时间 t 点输入网络的内容为$x(t)$，输出则记为 $y(t)$，而 $s(t)$代表网络的状态（即隐藏层状态）。输入向量 $x(t)$是由表示当前单词的向量 w 与时间 $t-1$时上下文层中神经元的输出 s 拼接而成的。随后，输入层、隐藏层和输出层的计算方式如下：

$$x(t) = w(t) + s(t-1) \tag{1}$$

$$s_j(t) = f\left(\sum_i x_i(t) u_{ji}\right) \tag{2}$$

$$y_k(t) = g\left(\sum_j s_j(t) v_{kj}\right) \tag{3}$$

其中 $f(z)$即为S形激活函数：

$$f(z) = \frac{1}{1+e^{-z}} \tag{4}$$

而 $g(z)$则是Softmax函数：

$$g(z_m) = \frac{e^{z_m}}{\sum_k e^{z_k}} \tag{5}$$

在初始化方面， $s(0)$可以设为一系列较小的数值，例如0.1——当处理的数据量很大时，初始化并非至关重要。在后续的时间步中，$s(t+1)$会复制 $s(t)$的内容。输入向量 $x(t)$使用1-of-N编码方式来表示时间 t 对应的单词以及之前的上下文信息，其向量大小等于词汇表 V的大小（实际应用中可能为 $30\,000 - 200\,000$）再加上上下文层的大小。上下文（即隐藏）层 s 的规模通常为 $30 - 500$ 个隐藏单元。根据我们的实验结果，隐藏层的规模应当与训练数据的量相匹配——面对海量数据时，就需要更大的隐藏层[1]。

这些网络会经过多个训练轮次进行训练，在每个轮次中，训练语料库中的所有数据都会按顺序被呈现。权重初始值会被设定为较小的数值（即均值为零、方差为0.1的高斯随机噪声）。在训练网络时，我们会采用标准的反向传播算法并结合随机梯度下降方法。初始的学习率值为 $\alpha = 0.1$。每完成一个训练轮次后，就会使用验证数据对网络进行测试。如果验证数据的对数似然值有所上升，训练就会在新的轮次中继续；若未观察到显著改进，则会在每个新轮次开始时将学习率 α 减半。一旦再次没有明显提升，训练便告结束。通常情况下，经过10到20个训练轮次后网络就能实现收敛。

在我们的实验中，即便使用规模极大的隐藏层，这些神经网络也不会出现严重的过训练现象——通过对网络施加正则化处理以惩罚过大的权重，也并未带来任何显著的改进效果。输出层 $y(t)$用于表示在给定前一个词 $w(t)$以及上下文的情况下，下一个词出现的概率分布。

[1] 因此，随着训练数据量的增加，训练出最优网络所需的时间并非呈线性增长，其增长速度会更快：词汇量的扩充会导致输入层和输出层的规模变大，同时更多的训练数据也会使得最优隐藏层的规模相应增大。

$s(t-1)$。Softmax函数能够确保这一概率分布是有效的，也就是说，对于任意一个词 m 而言，都有 $y_m(t) > 0$，同时对于 $\sum_k y_k(t)$来说则满足 $= 1$。

在每一个训练步骤中，都会根据交叉熵准则计算误差向量，随后再利用标准的反向传播算法来更新权重。

$$\text{error}(t) = \text{desired}(t) - y(t) \tag{6}$$

其中 $desired$ 是一个向量，采用1-of-N编码方式来表示在特定上下文中本应被预测出的单词，而$y(t)$则是网络实际输出的结果。

需要指出的是，在统计语言建模中，训练阶段与测试阶段的区别在于：在处理测试数据时模型不会被更新。因此，如果测试集中反复出现某个新的人名，即便该人名由已知的单词组成，它得到的概率也会一直非常低。可以认为，这类长期记忆不应存储在上下文单元的激活状态中（因为这些状态变化极快），而应存在于突触本身——也就是说，网络即使在测试阶段也应当持续训练。我们将这类模型称为动态模型。对于动态模型，我们会使用固定的学习率$\alpha = 0.1$。在训练阶段，所有数据会在多个训练周期中被多次呈现给网络，而动态模型在处理测试数据时仅更新一次。当然这并非最优解决方案，但正如我们后续将看到的，它已经足以让模型的困惑度较静态模型有显著下降。需要注意的是，这种修改与退避模型中的缓存技术十分相似，不同之处在于神经网络是在连续空间中进行学习的；所以如果“狗”和“猫”之间存在关联，那么在测试数据中“狗”频繁出现也会提升“猫”的出现概率。

通过动态更新，这些模型能够自动适配新的应用领域。但在语音识别实验中，历史数据实际上体现为识别器生成的假设，其中往往包含识别错误，这通常会导致缓存中的n-gram模型在ASR任务中的性能不佳 [2]。

此处介绍的训练算法也被称为带 $\tau = 1$的截断式随时间反向传播算法。由于该算法仅根据当前时间步计算的误差向量来更新网络权重，因此并非最优方案。为克服这一简化缺陷，人们通常会采用随时间反向传播算法（BPTT）（详情可参阅Boden的相关论述 [5] ）。

Bengio [3]与Schwenk [4] 所使用的反馈型神经网络与循环神经网络之间的一个主要区别在于，前者在训练前需要临时调整或选定的大量参数。对于基于循环神经网络的语言模型而言，仅需确定隐藏层（即上下文层）的规模即可；而对于反馈型网络，则需要调整将词语映射到低维空间的层的大小、隐藏层的规模以及上下文长度[2]。

2.1. 优化

为提升性能，我们会将训练文本中出现频率低于阈值的所有单词合并为一个特殊的稀有标记。随后即可按照如下方式计算单词概率：

$$P(w_i(t+1)|w(t), s(t-1)) = \begin{cases} \frac{y_{rare}(t)}{C_{rare}} & \text{if } w_i(t+1) \text{ is rare,} \\ y_i(t) & \text{otherwise} \end{cases} \tag{7}$$

[2] 本文并不打算对前馈网络与循环网络进行详细对比。不过在部分实验中，我们通过使用循环网络替代前馈网络，使得困惑度相比n-gram模型降低了近两倍。

Table 1: *Performance of models on WSJ DEV set when increasing size of training data.*

Model	# words	PPL	WER
KN5 LM	200K	336	16.4
KN5 LM + RNN 90/2	200K	271	15.4
KN5 LM	1M	287	15.1
KN5 LM + RNN 90/2	1M	225	14.0
KN5 LM	6.4M	221	13.5
KN5 LM + RNN 250/5	6.4M	156	11.7

where C_{rare} is number of words in the vocabulary that occur less often than the threshold. All rare words are thus treated equally, ie. probability is distributed uniformly between them.

Schwenk [4] describes several possible approaches that can be used for further performance improvements. Additional possibilities are also discussed in [10][11][12] and most of them can be applied also to RNNs. For comparison, it takes around 6 hours for our basic implementation to train RNN model based on Brown corpus (800K words, 100 hidden units and vocabulary threshold 5), while Bengio reports 113 days for basic implementation and 26 hours with importance sampling [10], when using similar data and size of neural network. We use only BLAS library to speed up computation.

3. WSJ experiments

To evaluate performance of simple recurrent neural network based language model, we have selected several standard speech recognition tasks. First we report results after rescoring 100-best lists from DARPA WSJ'92 and WSJ'93 data sets - the same data sets were used by Xu [8] and Filimonov [9]. Oracle WER is 6.1% for dev set and 9.5% for eval set. Training data for language model are the same as used by Xu [8].

The training corpus consists of 37M words from NYT section of English Gigaword. As it is very time consuming to train RNN LM on large data, we have used only up to 6.4M words for training RNN models (300K sentences) - it takes several weeks to train the most complex models. Perplexity is evaluated on held-out data (230K words). Also, we report results for combined models - linear interpolation with weight 0.75 for RNN LM and 0.25 for backoff LM is used in all these experiments. In further experiments, we denote modified Kneser-Ney smoothed 5-gram as KN5. Configurations of neural network LMs, such as RNN 90/2, indicate that the hidden layer size is 90 and threshold for merging words to rare token is 2. To correctly rescore n-best lists with backoff models that are trained on subset of data used by recognizer, we use open vocabulary language models (unknown words are assigned small probability). To improve results, outputs from various RNN LMs with different architectures can be linearly interpolated (diversity is also given by random weight initialization).

The results, reported in Tables 1 and 2, are by no means among the largest improvements reported for the WSJ task obtained just by changing the language modeling technique. The improvement keeps getting larger with increasing training data, suggesting that even larger improvements may be achieved simply by using more data. As shown in Table 2, WER reduction when using mixture of 3 dynamic RNN LMs against 5-gram with modified Kneser-Ney smoothing is about 18%. Also, perplexity reductions are one of the largest ever reported, almost 50% when comparing KN 5gram and mixture of 3 dynamic RNN LMs - actually, by mixing static and dynamic RNN LMs with larger learning rate used when processing testing data ($\alpha = 0.3$), the best perplexity result was 112.

Table 2: *Comparison of various configurations of RNN LMs and combinations with backoff models while using 6.4M words in training data (WSJ DEV).*

	PPL		WER	
Model	RNN	RNN+KN	RNN	RNN+KN
KN5 - baseline	-	221	-	13.5
RNN 60/20	229	186	13.2	12.6
RNN 90/10	202	173	12.8	12.2
RNN 250/5	173	155	12.3	11.7
RNN 250/2	176	156	12.0	11.9
RNN 400/10	171	152	12.5	12.1
3xRNN static	151	143	11.6	11.3
3xRNN dynamic	128	121	11.3	11.1

Table 3: *Comparison of WSJ results obtained with various models. Note that RNN models are trained just on 6.4M words.*

Model	DEV WER	EVAL WER
Lattice 1 best	12.9	18.4
Baseline - KN5 (37M)	12.2	17.2
Discriminative LM [8] (37M)	11.5	16.9
Joint LM [9] (70M)	-	16.7
Static 3xRNN + KN5 (37M)	11.0	15.5
Dynamic 3xRNN + KN5 (37M)	10.7	16.3[4]

All LMs in the preceding experiments were trained on only 6.4M words, which is much less than the amount of data used by others for this task. To provide a comparison with Xu [8] and Filimonov [9], we have used 37M words based backoff model (the same data were used by Xu, Filimonov used 70M words). Results are reported in Table 3, and we can conclude that RNN based models can reduce WER by around 12% relatively, compared to backoff model trained on 5x more data[3].

4. NIST RT05 experiments

While previous experiments show very interesting improvements over a fair baseline, a valid criticism would be that the acoustic models used in those experiments are far from state of the art, and perhaps obtaining improvements in such cases is easier than improving well tuned system. Even more crucial is the fact that 37M or 70M words used for training baseline backoff models is by far less than what is possible for the task.

To show that it is possible to obtain meaningful improvements in state of the art system, we experimented with lattices generated by AMI system used for NIST RT05 evaluation [13]. Test data set was NIST RT05 evaluation on independent headset condition.

The acoustic HMMs are based on cross-word tied-states triphones trained discriminatively using MPE criteria. Feature ex-

[3]We have also tried to combine RNN models and discriminatively trained LMs [8], with no significant improvement.

[4]Apparently strange result obtained with dynamic models on evaluation set is probably due to the fact that sentences in eval set do not follow each other. As dynamic changes in model try to capture longer context information between sentences, sentences must be presented consecutively to dynamic models.

表1：随着训练数据规模的扩大，各模型在*WSJ DEV*数据集上的性能表现。

模型	# 单词数	PPL	WER
KN5语言模型	200K	336	16.4
KN5语言模型 +RNN 90/2结构	200K	271	15.4
KN5语言模型	1M	287	15.1
KN5语言模型 +采用RNN 90/2结构	1M	225	14.0
KN5语言模型	6.4M	221	13.5
KN5语言模型 +采用RNN 250/5结构	6.4M	156	11.7

其中 C_{rare} 表示词汇表中出现频率低于阈值的单词数量。如此一来，所有稀有单词都会被同等对待，即它们的概率会在彼此之间均匀分配。

Schwenk [4] 介绍了几种可用于进一步提升性能的可行方法， [10][11][12] 中也探讨了其他可能性，而这些方法大多同样适用于循环神经网络。作为对比，在使用Brown语料库（包含80万词，100个隐藏单元以及5的词汇表阈值）的情况下，我们的基础实现训练基于循环神经网络的模型大约需要6小时；而Bengio在采用类似的数据和神经网络规模时，其基础实现需要113天，通过重要性采样 [10], 则仅需26小时。为加快计算速度，我们仅使用了BLAS数学库。

3. WSJ实验

为评估基于简单循环神经网络的语言模型的性能，我们选择了几项标准的语音识别任务。首先，我们展示了对DARPA WSJ'92和WSJ' 93数据集中的前100名结果列表进行重新评分后的测试结果——Xu [8] 和Filimonov [9]也曾使用过这些数据集。该语言模型的开发集Oracle词错误率为6.1%，评估集则为9.5%。用于训练该语言模型的训练数据与Xu [8]所使用的相同。

训练语料库源自English Gigaword中的NYT板块，共计3700万个单词。由于在海量数据上训练RNN大语言模型需要耗费大量时间，我们仅在训练RNN模型时使用了最多640万个单词（对应30万句文本）——训练最复杂的模型甚至需要数周时间。困惑度则是基于留出的测试数据（230万个单词）来评估的。此外，我们还报告了组合模型的测试结果：在所有实验中均采用了线性插值方法，其中RNN大语言模型的权重为0.75，而回退语言模型的权重则为0.25。在后续实验中，我们将改进型Kneser-Ney平滑五元语法称为KN5。至于那些基于循环神经网络构建的语言模型，比如RNN 90/2结构，其隐藏层节点数为90，且将单词合并为稀有标记的阈值设定为2。为了能够利用在识别器所用数据子集上训练得到的退避模型，正确地对n最佳列表进行重新评分，我们会使用开放词汇语言模型（对于未知单词则会赋予较低的概率值）。此外，通过将不同架构的多种RNN大语言模型的输出进行线性插值，也可以进一步提升模型性能（随机权重初始化则有助于增强结果的多样性）。

表1和表2中所呈现的实验结果，并不属于仅通过更换语言模型技术就能在《华尔街日报》相关任务上取得的最大改进幅度之列。随着训练数据的不断增加，改进效果也会持续提升，这表明只要使用更多数据，或许还能实现更大幅度的改进。如表2所示，采用3种动态RNN大语言模型的混合模型相比经过改进的Kneser-Ney平滑处理的5元语法模型，其词错误率可降低约18%。此外，困惑度下降幅度也是迄今为止报道过的最大值之一，当对比KNS 5元语法模型与这3种动态RNN大语言模型的混合模型时，困惑度降幅接近50%。

表2：在训练数据量为*640*万词（来自《华尔街日报》*DEV*数据集）的条件下，不同配置的*RNN*大语言模型以及与退避模型相结合的方案的对比情况。

	PPL		WER	
模型	RNN	RNN+KN	RNN	RNN+KN
KN5 – 基准模型	-	221	-	13.5
RNN 60/20	229	186	13.2	12.6
RNN 90/10结构	202	173	12.8	12.2
RNN 250/5结构	173	155	12.3	11.7
RNN 250/2结构	176	156	12.0	11.9
RNN 400/10结构	171	152	12.5	12.1
三倍RNN静态结构	151	143	11.6	11.3
三倍RNN动态结构	128	121	11.3	11.1

表3：使用不同模型所得的《华尔街日报》文本处理结果对比。需注意，这些循环神经网络模型仅基于*640*万词汇量进行训练。

模型	开发环境词错误率	测试环境词错误率
格点模型最优方案	12.9	18.4
基准模型KN5（3700万词汇量）	12.2	17.2
判别性语言模型 [8]（3700万词）	11.5	16.9
联合语言模型 [9]（7000万词）	-	16.7
静态3倍速率RNN KN5模型 +（3700万词）	11.0	15.5
动态3倍速率RNN KN5模型 +（3700万词）	10.7	16.3[4]

实际上，通过将静态RNN大语言模型与动态RNN大语言模型相结合，并在处理测试数据时采用更高的学习率（$\alpha = 0.3$），最终获得的最佳困惑度数值为112。

前述实验中的所有模型都仅使用640万词的数据进行训练，这一数值远低于其他研究者用于该任务的数据量。为了与Xu [8] 以及Filimonov [9], 的研究结果进行对比，我们采用了基于3700万词数据的退避模型（Xu使用了相同的数据量，而Filimonov则使用了7000万词数据）。相关实验结果列于表3中，从表中可以看出，与在5倍更多数据上训练的退避模型相比，基于循环神经网络的模型能够将词错误率相对降低约12%[3]。

4. NIST RT05实验

尽管先前的实验在基准水平上已经展现了非常显著的提升，但有人可能会指出，那些实验中使用的声学模型远未达到最先进水平，因此在这样的条件下取得改进可能比完善经过精心调优的系统要容易得多。更为关键的是，用于训练基准退避模型的大量文本——仅3700万或7000万词——远远不足以满足该任务的实际需求。

为了证明在最先进的系统中同样可以实现有意义的改进，我们使用了AMI系统生成的格点结构，并结合NIST RT05评估数据进行了实验 [13]。测试数据则为在独立耳机条件下的NIST RT05评估数据。

这些声学HMM是基于经过MPE准则进行区分性训练的、具有跨词绑定状态的三音素模型构建的。其特征提取

[3]我们还尝试将循环神经网络模型与经过判别式训练的机器学习模型 [8],相结合，但并未获得显著效果。[4]在评估集上使用动态模型所得到的反常结果，很可能是由于评估集中的句子并非按顺序排列所致。由于动态模型需要通过不断变化来捕捉句子之间的更长上下文信息，因此必须让这些句子依次呈现给模型。

Table 4: *Comparison of very large back-off LMs and RNN LMs trained only on limited in-domain data (5.4M words).*

Model	WER static	WER dynamic
RT05 LM	24.5	-
RT09 LM - baseline	24.1	-
KN5 in-domain	25.7	-
RNN 500/10 in-domain	24.2	24.1
RNN 500/10 + RT09 LM	**23.3**	23.2
RNN 800/10 in-domain	24.3	23.8
RNN 800/10 + RT09 LM	23.4	23.1
RNN 1000/5 in-domain	24.2	23.7
RNN 1000/5 + RT09 LM	23.4	22.9
3xRNN + RT09 LM	**23.3**	**22.8**

traction use 13 Mel-PLP's features with deltas, double and triple deltas reduced by HLDA to 39-dimension feature vector. VTLN warping factors were applied to the outputs of Mel filterbanks. The amount of training data was 115 hours of meeting speech from ICSI, NIST, ISL and AMI training corpora.

Four gram LM used in AMI system was trained on various data sources, see description in [13]. Total amount of LM training data was more than 1.3G words. This LM is denoted as RT05 LM in table 4. The RT09 LM was extended by additional CHIL and web data. Next change was in lowering cut-offs, e.g. the minimum count for 4-grams was set to 3 instead of 4. To train the RNN LM, we selected in domain data that consists of meeting transcriptions and Switchboard corpus, for a total of 5.4M words – RNN training was too time consuming with more data. This means that RNNs are trained on tiny subset of the data that are used to construct the RT05 and RT09 LMs. Table 4 compares the performance of these LMs on RT05.

5. Conclusion and future work

Recurrent neural networks outperformed significantly state of the art backoff models in all our experiments, most notably even in case when backoff models were trained on much more data than RNN LMs. In WSJ experiments, word error rate reduction is around 18% for models trained on the same amount of data, and 12% when backoff model is trained on 5 times more data than RNN model. For NIST RT05, we can conclude that models trained on just 5.4M words of in-domain data can outperform big backoff models, which are trained on hundreds times more data. Obtained results are breaking myth that language modeling is just about counting n-grams, and that the only reasonable way how to improve results is by acquiring new training data.

Perplexity improvements reported in Table 2 are one of the largest ever reported on similar data set, with very significant effect of on-line learning (also called dynamic models in this paper, and in context of speech recognition very similar to unsupervised LM training techniques). While WER is affected just slightly and requires correct ordering of testing data, on-line learning should be further investigated as it provides natural way how to obtain cache-like and trigger-like information (note that for data compression, on-line techniques for training predictive neural networks have been already studied for example by Mahoney [14]). If we want to build models that can really learn language, then on-line learning is crucial - acquiring new information is definitely important.

It is possible that further investigation into backpropagation through time algorithm for learning recurrent neural networks will provide additional improvements. Preliminary results on toy tasks are promising. However, it does not seem that simple recurrent neural networks can capture truly long context information, as cache models still provide complementary information even to dynamic models trained with BPTT. Explanation is discussed in [6].

As we did not make any task or language specific assumption in our work, it is easy to use RNN based models almost effortlessly in any kind of application that uses backoff language models, like machine translation or OCR. Especially tasks involving inflectional languages or languages with large vocabulary might benefit from using NN based models, as was already shown in [12].

Besides very good results reported in our work, we find proposed recurrent neural network model interesting also because it connects language modeling more closely to machine learning, data compression and cognitive sciences research. We hope that these connections will be better understood in the future.

6. Acknowledgements

We would like to thank Puyang Xu for providing WSJ data, and Stefan Kombrink for help with additional NIST RT experiments. The work was also partly supported by European project DIRAC (FP6-027787), Czech Ministry of Interior project No. VD20072010B16, Grant Agency of Czech Republic project No. 102/08/0707, Czech Ministry of Education project No. MSM0021630528 and by BUT FIT grant No. FIT-10-S-2.

7. References

[1] Mahoney, M. Text Compression as a Test for Artificial Intelligence. In AAAI/IAAI, 486-502, 1999

[2] Goodman Joshua T. (2001). A bit of progress in language modeling, extended version. Technical report MSR-TR-2001-72.

[3] Yoshua Bengio, Rejean Ducharme and Pascal Vincent. 2003. A neural probabilistic language model. Journal of Machine Learning Research, 3:1137-1155

[4] Holger Schwenk and Jean-Luc Gauvain. Training Neural Network Language Models On Very Large Corpora. in Proc. Joint Conference HLT/EMNLP, October 2005.

[5] Mikael Bodén. A Guide to Recurrent Neural Networks and Backpropagation. In the Dallas project, 2002.

[6] Yoshua Bengio and Patrice Simard and Paolo Frasconi. Learning Long-Term Dependencies with Gradient Descent is Difficult. IEEE Transactions on Neural Networks, 5, 157-166.

[7] Jeffrey L. Elman. Finding Structure in Time. Cognitive Science, 14, 179-211

[8] Puyang Xu and Damianos Karakos and Sanjeev Khudanpur. Self-Supervised Discriminative Training of Statistical Language Models. ASRU 2009.

[9] Denis Filimonov and Mary Harper. 2009. A joint language model with fine-grain syntactic tags. In EMNLP.

[10] Bengio, Y. and Senecal, J.-S. Adaptive Importance Sampling to Accelerate Training of a Neural Probabilistic Language Model. IEEE Transactions on Neural Networks.

[11] Morin, F. and Bengio, Y. Hierarchical Probabilistic Neural Network Language Model. AISTATS'2005.

[12] Tomáš Mikolov, Jiří Kopecký, Lukáš Burget, Ondřej Glembek and Jan Černocký: Neural network based language models for highly inflective languages, In: Proc. ICASSP 2009.

[13] T. Hain. et al., "The 2005 AMI system for the transcription of speech in meetings," in Proc. Rich Transcription 2005 Spring Meeting Recognition Evaluation Workshop, UK, 2005.

[14] Mahoney, M. 2000. Fast Text Compression with Neural Networks. In Proc. FLAIRS.

表4：仅在*540*万字的有限领域内数据上训练的超大退避型*RNN*大语言模型与普通*RNN*大语言模型的对比。

模型	静态WER值	动态WER值
RT05语言模型	24.5	-
RT09语言模型——基准模型	24.1	-
KN5模型在领域内应用场景	25.7	-
RNN 500/10模型在领域内应用场景	24.2	24.1
RNN 500/10 + RT09语言模型	**23.3**	23.2
RNN 800/10模型在领域内应用场景	24.3	23.8
RNN 800/10 + RT09语言模型	23.4	23.1
RNN 1000/5模型在领域内应用场景	24.2	23.7
循环神经网络1000/5 + RT09语言模型	23.4	22.9
3倍速率RNN +RT09语言模型	**23.3**	**22.8**

采用13个带差分的Mel-PLP特征，再通过HLDA算法将双倍和三倍差分值压缩为39维的特征向量。同时，还对Mel滤波器组的输出应用了VTLN变形因子。所用训练数据共计115小时的会议语音，这些语音来源于ICSI、NIST、ISL以及AMI这四个训练语料库。

AMI系统中使用的4元语法单元类型的语法语言模型是基于多种不同的数据源进行训练的，具体说明见 [13]。该语言模型的训练数据总量超过了1.3亿词。在表4中，这种语言模型被标记为RT05语言模型。而RT09语言模型则是通过引入额外的CHIL和网页数据进一步扩展而来的。接下来的改进措施是降低相关阈值，例如4元语法单元的最小出现次数从4改为3。在训练循环神经网络类型的语言模型时，我们选用了由会议记录转录文本和交换台语料库组成的领域数据，总词数为540万词——因为数据量过大，使用更多数据训练循环神经网络会耗费过多时间。这意味着用于构建RT05和RT09语言模型的循环神经网络实际上只是在这些数据中的极小子集上经过训练的。表4对比了这些语言模型在RT05任务上的性能表现。

5. 结论与未来工作

在所有实验中，循环神经网络的表现均显著优于现有的各类退避模型，即便在退避模型所使用的训练数据量远多于RNN大语言模型的情况下也是如此。在基于《华尔街日报》数据的实验中，当模型使用相同量的训练数据时，词错误率可降低约18%；而若退避模型使用的训练数据量是RNN模型的5倍，则词错误率的下降幅度可达12%。针对NIST RT05实验，我们得出结论：仅需540万条领域内数据即可训练出性能超越那些依靠海量数据训练的大型退避模型的模型。这些研究结果打破了"语言模型构建仅依赖于n-gram统计"以及"提升模型性能的唯一途径就是获取更多训练数据"之类的误区。

表2中所显示的困惑度下降幅度是同类数据集上迄今记录到的最大降幅之一，这充分体现了在线学习（本文中也称为动态模型，在语音识别领域其与无监督语言模型训练技术极为相似）的巨大作用。虽然词错误率受其影响相对较小，且还需要确保测试数据的正确排序，但依然有必要进一步研究在线学习技术，因为它为获取类似缓存和触发机制的功能提供了自然可行的方法（值得一提的是，例如Mahoney [14]就已经对用于训练预测型神经网络的在线压缩技术进行了相关研究）。如果我们希望构建真正能够学习语言的模型，那么在线学习就至关重要——持续获取新信息无疑具有极其重要的意义。

进一步研究用于训练循环神经网络的随时间反向传播算法，有望带来更多改进。

在简单测试任务上的初步结果相当令人满意。不过，简单循环神经网络似乎无法捕捉真正长距离的上下文信息，因为即便使用该算法训练的动态模型，缓存模型依然能提供补充信息。相关解释详见 [6]。

由于我们的研究并未做出任何针对特定任务或语言的假设，因此几乎可以毫不费力地将基于循环神经网络的模型应用于各类使用回退式语言模型的场景，比如机器翻译和光学字符识别。正如 [12]中所展示的，那些涉及屈折语或词汇量极大的语言的任务，尤其能从基于神经网络的模型中受益。

除了研究所得到的优异成果之外，我们认为所提出的循环神经网络模型极具价值，因为它将语言建模与机器学习、数据压缩以及认知科学研究更紧密地联系在了一起。我们期望未来这些关联能得到更深入的理解。

6. 致谢

我们要感谢徐濮阳提供了《华尔街日报》数据，同时也感谢斯特凡·科姆布林克在协助开展额外的NIST语料库实验方面给予的帮助。此外，这项工作还得到了DIRAC欧洲项目（FP6-027787）、捷克内政部VD20072010B16号项目、捷克共和国资助机构102/08/0707号项目、捷克教育部MSM0021630528号项目，以及BUT FIT项目FIT-10-S-2号项目的部分支持。

7. 参考文献

[1] Mahoney, M.《将文本压缩作为人工智能的测试》。载于AAAI/IAAI会议论文集，第486-502页，1999年[2] Goodman Joshua T.（2001）。《语言模型领域的些许进展——扩展版》，技术报告MSR-TR-2001-72[3] Yoshua Bengio、Rejean Ducharme与Pascal Vincent。2003。《一种神经概率语言模型》，《机器学习研究杂志》，第3卷，第1137-1155页[4] Holger Schwenk与Jean-Luc Gauvain。《在极大规模语料库上训练神经网络语言模型》，载于2005年HLT/EMNLP联合会议论文集[5] Mikael Bod´en。《循环神经网络与反向传播指南》，载于2002年的达拉斯项目相关资料[6] Yoshua Bengio、Patrice Simard与Paolo Frasconi。《利用梯度下降法学习长期依赖关系十分困难》，《IEEE神经网络交易杂志》，第5卷，第157-166页[7] Jeffrey L. Elman。《时间中的结构探索》，《认知科学》，第14卷，第179-211页[8] 徐濮阳、Damianos Karakos与Sanjeev Khudanpur。《统计语言模型的自监督判别训练》，2009年ASRU会议[9] Denis Filimonov与Mary Harper。2009。《一种带有细粒度句法标签的联合语言模型》，载于EMNLP会议论文集[10] Bengio, Y.与Senecal, J.-S。《利用自适应重要性采样加速神经概率语言模型的训练》，《IEEE神经网络交易杂志》[11] Morin, F.与Bengio, Y。《分层概率神经网络语言模型》，AISTATS'2005会议论文集[12] Tom´aˇs Mikolov、Jiˇr´ı Kopeck´y、Luk´aˇs Burget、Ondˇrej Glembek与Janˇ Cernock´y:《面向高度屈折语言的基于神经网络的语言模型》，载于2009年ICASSP会议论文集[13] T. Hain等人，"2005年用于会议语音转写的AMI系统"，载于2005年在英国召开的2005年Rich转录春季会议识别评估研讨会论文集[14] Mahoney, M.。2000。《利用神经网络实现快速文本压缩》，载于FLAIRS会议论文集。